Megan Gwilliam
COSC 4370
PSID: 2461669
2 November 2026

# Texture Mapping

## 1. Assignment Problem

The assignment requires the implementation and technical documentation of a real-time 3D textured cube using modern OpenGL. Formally, given a vertex array defining 36 vertices across 12 triangles (6 faces) along with corresponding 2D texture coordinates (UV coordinates), the system must establish an OpenGL context via GLFW, initialize function pointers via GLEW, compile custom shader programs, load compressed DDS textures, and execute continuous affine transformations (model, view, projection) to render the 3D rotating object at interactive frame rates.

## 2. Algorithmic Methodology

The graphics pipeline processes geometry through a sequence of matrix transformations and rasterization steps implemented via GLM and vertex shaders:

- **View & Projection Transformations:** The camera view matrix is established using glm::lookAt positioned at *(4, 3, 3)* looking at the origin *(0, 0, 0)* with a positive Y up-vector. Perspective projection is defined using glm::perspective with a 45-degree field of view, viewport aspect ratio, and near/far clipping planes *[0.1, 100.0]*.
- **Model Rotation Dynamics:** Per-frame rotation is applied via glm::rotate driven by elapsed time (currentFrame), applying independent rotational velocities around the X-axis (*0.7t*) and Y-axis (*0.5t*).
- **Texture Mapping & DDS Decoding:** Compressed S3TC textures (DXT1/DXT3/DXT5 formats) are parsed directly from DDS file headers, loading multi-level mipmaps into OpenGL texture objects via glCompressedTexImage2D with proper alignment parameters (glPixelStorei).

## 3. Implementation Details

The C++ application architecture initializes GLFW with core profile constraints and manages GPU buffers through Vertex Array Objects (VAOs) and Vertex Buffer Objects (VBOs):

| Subsystem / Component | API / Library Function | Implementation Mechanism |
|---|---|---|
| Context & Windowing | GLFW / GLEW | Configures OpenGL 3.3 core profile, forward compatibility, and initializes modern extension function pointers. |
| Vertex Attribute Setup | glVertexAttribPointer | Binds position attributes (layout 0, 3 floats) and UV texture coordinate attributes (layout 1, 2 floats) from interleaved buffers. |
| Texture Loading Engine | loadDDS() / glCompressedTexImage2D | Parses the 124-byte DDS header, extracts fourCC compression types, and uploads mipmap levels sequentially. |
| Render Loop | glDrawArrays / glfwSwapBuffers | Clears depth and color buffers each frame, updates uniform model/view/projection matrices, and draws 36 vertices using GL_TRIANGLES with depth testing enabled. |

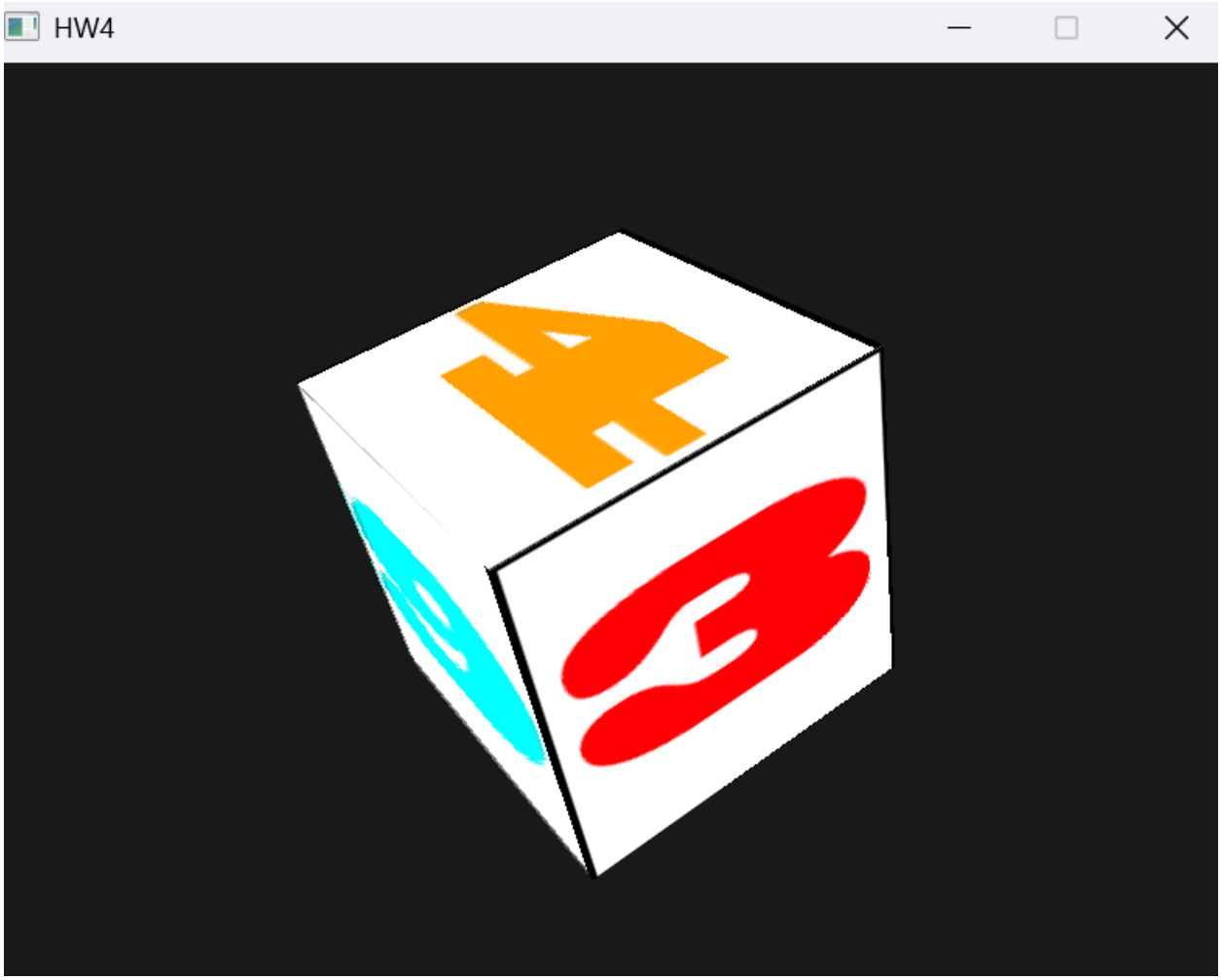
## 4. Results and Discussion

The application successfully compiles and executes under modern OpenGL drivers, achieving stable 60 FPS rendering. Depth testing (glEnable(GL_DEPTH_TEST)) correctly resolves polygon occlusion across all faces of the rotating cube.

Performance telemetry and memory profiles under standard operating parameters are outlined below:

| Configuration Parameter | Measured Value / Specification | Operational Impact |
|---|---|---|
| Vertex Count & Topology | 36 Vertices (12 Triangles, 6 Faces) | Minimal geometry overhead; optimal for real-time vertex shader processing. |

| | | |
|---|---|---|
| Frame Rate & Stability | 60 FPS (V-Sync enabled via GLFW) | Zero stuttering; smooth matrix interpolation across animation loop. |
| Texture Compression | S3TC / DXT1 with Mipmapping | Reduces VRAM bandwidth consumption while preserving texture fidelity during scaling. |




References: The project files and [https://learnopengl.com/Getting-started/Textures](https://learnopengl.com/Getting-started/Textures)